

MANTIS — Modular Agent Network Testbed for Instrumentation and Security in Multi-Agent Banking Systems

1 st Given Name Surname dept. name of organization (of Aff.) name of organization (of Aff.) City, Country email address or ORCID	2 nd Given Name Surname dept. name of organization (of Aff.) name of organization (of Aff.) City, Country email address or ORCID	3 rd Given Name Surname dept. name of organization (of Aff.) name of organization (of Aff.) City, Country email address or ORCID
4 th Given Name Surname dept. name of organization (of Aff.) name of organization (of Aff.) City, Country email address or ORCID	5 th Given Name Surname dept. name of organization (of Aff.) name of organization (of Aff.) City, Country email address or ORCID	6 th Given Name Surname dept. name of organization (of Aff.) name of organization (of Aff.) City, Country email address or ORCID

Abstract—Multi-agent systems built on large language models (LLMs) are increasingly deployed in banking for transaction monitoring, operations planning, and end-of-day reconciliation, yet there is no standard, reusable way to evaluate such systems under adversarial conditions without either rewriting the framework core or the business workflow logic being tested. We present MANTIS (Modular Agent Network Testbed for Instrumentation and Security), a configuration-driven testbed built around a banking multi-agent system spanning front-, mid-, and back-office workflows. MANTIS instruments five fixed control points (input, agent, interaction, tool, and output) through a hook bus that lets an experimenter declare an attack or failure condition entirely in YAML, register it as a plugin against the same interface used by every other plugin, and leave the underlying banking agents unmodified. Every run produces a signed manifest (configuration hash, seed, and environment metadata), a complete OpenTelemetry- and MLflow-integrated trace, and an automated evaluation across seven scored dimensions, including tool-use correctness, terminal-state outcome, artifact integrity, and whether a configured attack’s effect is corroborated by ground truth rather than merely having fired. We describe the architecture, the banking runtime adapter covering 31 agents and 20 tools (19 banking-domain tools plus the framework’s own routing tool) across three domains, and the observability and evaluation pipeline. We report a case study in why verifying a security experiment has to inspect the emitted trace end to end rather than trust a clean process exit or an isolated “attack fired” flag: doing so surfaced two independent, previously-undetected defects in the hook bus’s own plugin-dispatch aggregation and in how a mutated tool argument was applied back onto the agent framework’s real execution path, present since the mechanism was first written, under which every attack plugin’s payload mutation had recorded as having occurred while having no effect on real agent behavior. We diagnose both directly against the underlying framework’s source, fix them, and re-verify all five attack and failure configurations live against a real model across five repeated trials each, observing genuine behavioral

divergence – a bypassed compliance review reproduced in every trial, a redirected wire transfer, a misdirected ledger post – caught automatically by the corresponding ground-truth evaluator, and a scenario-design gap in one attack’s fixture that a single trial could not have surfaced. A tool-calling-capable mock model isolates instrumentation overhead from LLM sampling latency, giving a repeated-trial (n=10) direct measurement with a median of 0.19% of run duration, corroborated by a contention-controlled end-to-end comparison of comparable magnitude, together with an independent concurrency-scaling sweep and full three-domain volume-scaling depth.

Index Terms—multi-agent systems, adversarial security, large language models, banking, observability, benchmarking, red teaming

I. Introduction

Large language model (LLM) agents are moving from single-turn assistants into orchestrated multi-agent systems that carry out multi-step business processes. In banking specifically, recent work has proposed multi-agent frameworks for anomaly detection in financial markets [1] and for credit assessment [2], each decomposing a task across specialized agents – a pattern that mirrors how a bank actually organizes front-, mid-, and back-office work today.

That same decomposition is also where new security risk enters. Safety properties established for a single chat model do not transfer to a system where behavior emerges from message passing, tool invocation, and routing decisions across multiple agents [5]. Recent attacks demonstrate that inter-agent communication itself is an exploitable surface: prompt injection can propagate from one agent to another like an infection [3], and an adversary positioned between two agents can intercept and rewrite the messages passing between them without ever touching either agent’s model weights or prompts [4]. A survey of

agentic security threats and defenses concludes that most current defenses evaluated in isolation can be bypassed by an adaptive attacker [5], underscoring that security evaluation for these systems needs to happen at the level of the deployed multi-agent system, not the underlying model.

Despite this, there is no widely available testbed that lets a researcher take a realistic, domain-specific multi-agent system and run controlled adversarial experiments against it without either reimplementing the orchestration framework or editing the business logic under test. General-purpose red-teaming benchmarks for LLM agents exist [7], but they are typically domain-agnostic and are not designed to instrument an existing, unmodified business workflow at fixed points chosen for their security relevance.

This paper presents MANTIS, a modular, configuration-driven testbed built directly on top of a banking multi-agent system spanning front-office (transaction monitoring, fraud review, compliance, customer service), mid-office (operations planning, representative assist), and back-office (end-of-day reconciliation and settlement) workflows. Our contributions are:

- A hook bus architecture that exposes five fixed control points – input, agent, interaction, tool, and output – around an existing agent orchestration pipeline, so that an attack or failure plugin can observe, mutate, delay, or block a single interaction without any change to the banking agents themselves.
- A banking runtime adapter covering three domains, 31 agents, and 20 tools (19 banking-domain tools – customer lookup, transaction execution, ledger posting, reconciliation, and reporting – plus the underlying agent framework’s own routing tool), introspected live from the running system rather than maintained as a separate, driftable inventory.
- Five attack and failure configurations spanning all three domains – prompt injection, inter-agent message spoofing, route confusion, tool-parameter mutation (shipped against both a front-office funds transfer and a back-office ledger post), and a reliability-failure family (delay, timeout, malformed result) – registered through a single plugin interface, with tool- and interaction-targeted attacks additionally carrying a machine-readable expected-tool and expected-terminal-state ground truth for automated grading, re-verified live against a real model to produce a genuine, ground-truth-detected behavioral divergence rather than merely an “attack fired” flag.
- A standard observability pipeline that maps every hook dispatch to a typed trace event (experiment, workflow, agent, interaction, tool, security, and evaluation families, the security family itself split into an adversarial `ATTACK_INJECTED` event and an operational `ANOMALY` event for reliability failures), exports to OpenTelemetry and MLflow, and writes

portable JSONL traces annotated with banking-specific semantics such as risk level, data sensitivity, and financial side effect.

- A reproducible evaluation and benchmarking CLI that hashes every configuration, records environment metadata (interpreter version, dependency versions, and source commit) alongside each run, scores seven dimensions per run – trace completeness, tool-use correctness, terminal-state outcome, hook-point coverage, banking-domain coverage, artifact integrity (a re-hash of the trace cross-checked against the manifest), and attack ground truth (whether a configured attack fired and whether its effect is corroborated by the run’s own expected-tools/terminal-state baseline) – and sweeps a directory of experiment configurations into a single comparative report.

The remainder of this paper is organized as follows. Section II motivates the problem and situates MANTIS relative to prior work on agentic security and Zero Trust access control. Section III describes the MANTIS architecture in detail. Section IV reports preliminary, live-verified evidence for each component and describes the full evaluation campaign this testbed is designed to run. Section V concludes and outlines future work.

II. Background and Motivation

A. Multi-Agent LLM Systems in Banking

Multi-agent decomposition is a natural fit for banking workloads because the workloads themselves are already organized around specialized roles: a transaction monitoring function, a fraud investigation function, a compliance sign-off, and a decision. Prior work applying LLM-based multi-agent frameworks to financial anomaly detection [1] and credit assessment [2] demonstrates that this decomposition is being actively explored in the financial domain, typically by assigning each specialized role to its own agent and coordinating them through a router or planner agent.

B. Security Risks Specific to Multi-Agent Systems

Three properties of multi-agent LLM systems create risk that does not exist in a single-agent deployment. First, agents routinely ingest content produced by other agents, not only by end users, so an attacker who compromises or spoofs one agent’s output can influence every downstream agent that consumes it [3]. Second, routing and hand-off decisions are themselves attack surfaces: an adversary that can intercept or relabel an inter-agent message can redirect a request to the wrong specialist or bypass a required review step entirely [4]. Third, tool invocation – the mechanism by which an agent actually takes action in the world, such as executing a transfer or posting a ledger entry – can be mutated in-flight, which is a materially different threat from a model simply producing an incorrect answer. A broad survey of agentic security confirms that these classes of attack generalize across frameworks and

that isolated, model-level defenses are insufficient against an adversary operating at the orchestration layer [5], [6].

C. The Gap: A Reusable, Domain-Grounded Testbed

General red-teaming benchmarks for LLM agents [7] provide valuable, reusable attack scaffolding, but they evaluate agents against synthetic or generic tasks rather than an existing production-shaped business workflow, and they are not designed around the constraint that the business logic under test must remain completely unmodified. This second constraint matters in practice: a testbed that requires editing the fraud-review agent’s prompt or control flow to add an experiment cannot be trusted to measure that agent’s real behavior. MANTIS is designed specifically to satisfy both properties at once – a realistic, domain-grounded multi-agent system, instrumented non-invasively.

D. Zero Trust as a Complementary, Deferred Direction

Zero Trust architecture – the principle of never trusting a request implicitly, regardless of its origin, and mediating every action through explicit policy [8] – is increasingly being proposed as a governance layer for agentic AI systems specifically, including fine-grained, identity-based access control for individual agents [9]. MANTIS’s inventory of agents and tools, and its per-control-point metadata, are exactly the inputs a Zero Trust policy decision point would need. We treat Zero Trust enforcement as a deliberately separate, later research contribution: this paper’s scope is limited to making that inventory and control-point metadata available through a documented interface, not to implementing enforcement.

III. Methodology

A. System Overview

Figure 1 shows the MANTIS pipeline. An experiment is declared entirely in a YAML configuration, which an orchestrator loads to establish a run identifier, a seed, and the experiment’s lifecycle. The orchestrator drives the banking workflow through a hook bus exposing five control points. An attack or failure plugin registers against the hook bus at one of those five points; every dispatch also passes through an observability plugin that is guaranteed to observe – though never able to reverse – whatever an attack plugin has already done to the payload in that same dispatch. Beneath the hook bus, a banking runtime adapter presents one stable interface (inventory, workflow lookup, and execution) over the front-, mid-, and back-office agents, which are otherwise unmodified. Every event is written to a standard observability pipeline, which an evaluator and benchmark runner then consume to produce scored, reproducible results.

B. Banking Runtime Adapter and Domain Coverage

The banking runtime adapter presents a single interface – `inventory()`, `workflows()`, and `build()` – over three

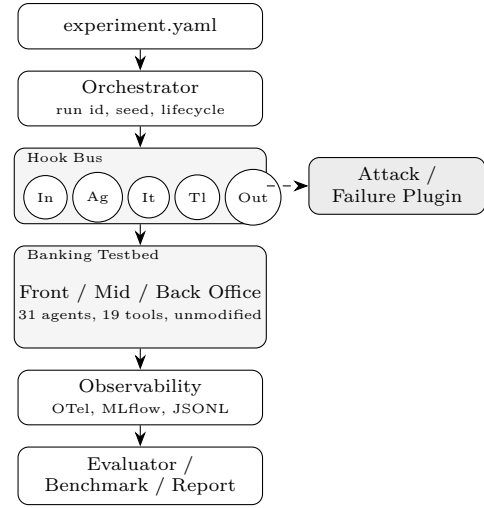


Fig. 1. MANTIS pipeline. Every run passes through the same five-point hook bus regardless of banking domain; an attack or failure plugin attaches to the bus and is dispatched alongside the standard observability plugin, without touching the banking agents beneath it.

domains. The front office covers real-time transaction monitoring, fraud detection, compliance, and a customer-service chatbot, comprising 11 agents. The mid office covers operations planning and representative-assist workflows, comprising 12 agents. The back office covers end-of-day reconciliation and settlement, comprising 8 agents. Across the three domains, agents call 19 distinct banking tools, from read operations such as retrieving a customer’s transaction history to write operations with a direct financial side effect, such as executing a transfer or posting a ledger update; the adapter’s raw inventory reports 20 tools in total, the twentieth being the underlying agent framework’s own `transfer_to_agent` routing tool rather than a banking-domain tool. The inventory returned by the adapter is introspected live from the tool modules and the agent registry at call time, rather than maintained as a hand-authored list, so that the reported system surface cannot silently drift from the deployed system.

C. Hook Bus and Five Control Points

The hook bus dispatches a typed context object – carrying a run identifier, trace identifier, workflow identifier, the control-point stage, a source and target, a payload, and free-form metadata – to every registered plugin whose declared stages match the current control point. A plugin returns one of five actions: `continue`, `mutate`, `skip`, `deny`, or `error`. When a plugin mutates the payload, the mutated version is passed to the next plugin in the chain, and the fact that a mutation (or `skip`, `deny`, or `error`) occurred is recorded in the context metadata so that a plugin registered later in the same dispatch – notably the observability plugin – can distinguish a genuinely adversarial event from an ordinary one, rather than simply observing an already-mutated payload indistinguishable from a normal

one. The five control points – input, agent, interaction, tool, and output – are chosen to bracket every externally observable transition in the agent lifecycle: raw input before any agent reasons about it, an agent’s invocation of the underlying model, the message an agent sends to or receives from that model, a tool call and its result, and the final output returned to the caller. A coverage report records, per run, how many times each of the ten before/after hook points fired and which plugins acted on each, which is itself a queryable artifact for confirming that a given experiment configuration actually exercised the control point it claimed to; § IV-B reports scoring this coverage as an eighth evaluator dimension. A prior version of the dispatch loop returned immediately upon the first mutate, skip, deny, or error action – reasonable for skip/deny/error, which should stop the chain, but not for mutate, which should continue it, and the loop’s own final fallback additionally collapsed the aggregate result back to continue whenever the plugin dispatched last (the observability plugin, by design, so that it can observe every attack’s effect) did not itself act. § IV-B reports the consequence of that defect and its fix in detail, since it is the reason no attack plugin’s payload mutation reached the framework’s real execution path prior to this work, independent of any individual plugin’s own targeting logic.

D. Attack and Failure Plugins

Four attack plugins are implemented against the same interface. Prompt injection and message spoofing both intercept the interaction control point – an agent’s outgoing call to the underlying model – and mutate the request content directly: prompt injection appends an adversarial instruction to the target agent’s outgoing message before it reasons about a transaction, and message spoofing substitutes a fabricated message, for example a false compliance-clearance signal, attributed to an impersonated sender, into a specific agent’s outgoing call. Both plugins originally targeted the input and agent control points respectively; § IV-B reports that neither of those two control points ever writes a mutated payload back onto the framework’s real invocation in this integration, so a plugin registered there could report having fired with zero effect on real agent behavior, and both were moved to the interaction stage, the only stage wired for real effect on an outgoing model request. Route confusion intercepts a routing tool call and forces execution to a different downstream agent than the one the orchestration logic selected – concretely, diverting a suspicious-transaction review into the customer-service chatbot workflow, bypassing fraud detection and compliance review entirely. Tool-parameter mutation intercepts a tool call directly and rewrites its arguments, for example redirecting the destination account on a funds transfer, before the call reaches the banking backend. This fourth plugin is shipped as two configurations rather than one: the front-office configuration above, and a back-office configuration that

instead mutates the destination batch identifier of an end-of-day ledger-posting call, redirecting a validated batch’s ledger write onto a second, unvalidated batch already present in the domain data – a confused-deputy attack on settlement rather than on payments, and, together with prompt injection’s and message spoofing’s front- and mid-office targets, giving five shipped attack configurations that span all three banking domains. A sixth plugin family implements ordinary reliability failures – delay, timeout, and malformed results – emitted as an ANOMALY security event rather than ATTACK_INJECTED (§ III-E), so that an evaluator can separate a security-caused anomaly from a routine fault by event type rather than by inference. Every attack plugin declares the control points and parameters it supports; the tool- and interaction-targeted plugins additionally specify their expected tool calls and, where applicable, expected terminal state through the evaluation schema, so that mantis --evaluate can report automatically whether the workflow reached the point the attack targets and whether its outcome diverged from the undisturbed baseline, rather than requiring manual inspection of the transcript to notice that it did not – a distinction Section IV shows is not merely hypothetical.

E. Observability Pipeline

Every hook dispatch is mapped to one of seven typed event families: experiment (run-level start/end, configuration hash, seed, and status), workflow (start, end, and routing decisions, tagged with business domain and terminal outcome), agent (start, end, and error, with latency), interaction (message sent to, received from, or mutated en route to the underlying model), tool (call, result, and error, tagged with operation type, risk level, data sensitivity, and financial side effect), security (an ATTACK_INJECTED event whenever an adversarial plugin took a non-trivial action, or an ANOMALY event for an operational reliability failure, each carrying the stage, target, plugin, and observed effect), and evaluation (the score each of the seven automated evaluators assigned, with a reference to its evidence, appended to the trace after mantis --evaluate runs so trace and evaluation artifacts share the same run identifier). Every event is written as a line of portable JSON so that trace analysis never depends on a specific backend, in addition to being exported as OpenTelemetry spans and as an MLflow run carrying the same configuration hash and seed for cross-referencing.

F. Evaluation and Benchmarking Framework

An automated evaluator reads a run’s trace and reports seven scores. Trace completeness checks whether the mandatory workflow, agent, and tool events are present. Tool-use correctness checks whether every tool the configuration marked as expected was actually called and no tool marked as forbidden was called. Workflow outcome checks whether the terminal state the run actually reached – inferred from the last business-significant

tool invoked, such as a manual-review submission or a transfer execution – matches the configuration’s expected terminal state. Hook-point coverage scores the fraction of the ten before/after control points (§ III-C) that fired at least once. Banking-domain coverage scores the fraction of the three domains exercised across a run or campaign. Artifact integrity re-hashes the trace file and cross-checks its run identifier against the manifest, so a truncated or tampered trace is detectable rather than silently trusted. Attack ground truth is specific to a run with a configured attack: it checks whether the plugin’s security event actually appears in the trace and whether the observed tool use or terminal state diverges from the configuration’s own expected-tools/expected-terminal-state baseline – distinguishing a plugin that fired but had no measurable effect from one whose effect is corroborated by ground truth, and reporting not-applicable rather than a score for a baseline run with no attack configured. A benchmark runner executes a configuration a specified number of times at a specified concurrency, aggregating latency percentiles and throughput, and additionally reports cumulative subprocess CPU time, peak resident memory, and mean trace event count per run; a campaign command sweeps every configuration in a directory, isolates each run’s artifacts, and produces one consolidated, human-readable report scoring every experiment side by side.

G. Reproducibility

Every run manifest records a SHA-256 hash of the fully normalized configuration, the configured random seed, and, as of this work, the Python interpreter version, the installed version of every framework-critical dependency, and the source repository’s current commit, so that a reported result can always be traced back to the exact code and environment that produced it. Because the underlying agents are backed by a live LLM, an identical seed narrows but does not eliminate run-to-run variance in exact wording; evaluators are accordingly designed to score bounded, semantic properties – which tools were called, what terminal state was reached – rather than exact text matches.

IV. Experimental Results

A. Implementation Status

All components described in Section III are implemented and have been exercised end to end against a live backend and, in this revision, a live model. The offline automated test suite comprises 92 unit and contract tests plus 49 baseline-regression tests (141 total), plus 13 tests for the standalone banking backend and 3 for the MCP tool server, all of which run in seconds without a live model or backend. A continuous-integration workflow runs this offline suite – with the banking backend started first, so no test’s pass/fail status depends on whichever tool a mock model happens to call first – plus a live execution of one attacked scenario against the real multi-agent pipeline on

every push, using the mock model described in § IV-C so that the check needs no API credential; a failure of that check means an attack plugin stopped actually firing, not merely that a process exited non-zero. Two of the 92 unit tests are pytest-level (not shell-script) end-to-end checks: one runs a baseline workflow and one runs an attacked workflow through the real CLI/ADK/MCP pipeline under the mock model and asserts a genuine `ATTACK_INJECTED` event followed by a corroborating `attack_ground_truth` evaluation. This section reports evidence gathered against a live backend and, for every attack and failure configuration, a live model; § IV-E describes what remains before this evidence is a statistically established evaluation campaign at scale rather than a targeted, small-sample ($n = 5$ per configuration) one.

B. Live-Verified Evidence and a Root-Cause Investigation

Hook coverage was measured across representative front-, mid-, and back-office runs; all ten before/after hook points recorded at least one hit in every run, confirming that no control point is silently unreachable for the workflows exercised. An earlier iteration of this evaluation found and fixed a single-argument-name mismatch in the route-confusion plugin (it matched `target_agent/agent_id`, where the framework’s real transfer tool takes `agent_name`) and reported live re-verification of that fix as pending API access. With that access now available, re-running the corrected plugin live surfaced that the fix, while itself correct, was not sufficient: the corrected plugin still recorded an `ATTACK_INJECTED` event on every run while the workflow continued to reach its original, unattacked destination every time. Tracing this required reading the hook bus’s own dispatch loop and the underlying agent framework’s plugin-callback contract directly, and surfaced two independent defects, present since the mechanism was first written, neither specific to route confusion:

- 1) Dispatch aggregation. The hook bus threads a mutated payload correctly from one plugin to the next within a single dispatch, but its final return value reported the aggregate action as `continue` whenever the last plugin dispatched – the observability plugin, by design, so it can observe every attack’s effect (§ III-C) – did not itself mutate anything. Since the observability plugin is registered after every attack plugin in every run, this meant the aggregate result of every dispatch that contained a mutation still reported `continue`, and the code path that applies a dispatch’s result to the real agent framework call only did so when the reported action was exactly `mutate`.
- 2) Tool-argument application. Independent of the above, the framework’s own plugin contract treats a non-None return from a before-tool callback as a substitute response that skips calling the real tool entirely, not as a modified-arguments instruction to

call it with. The integration layer was returning the mutated payload directly, which – once the first defect was corrected far enough to see it – caused the framework to skip calling the real `transfer_to_agent` function altogether and fabricate a response in its place, rather than calling the real function with the mutated destination.

The same root-cause read of the framework’s real request objects also explained why message spoofing and prompt injection needed to move control points (§ III-D): the object the framework actually passes into a model call exposes only a role and a list of parts, each optionally carrying text – verified directly against the installed package – not the content/sender attributes an earlier version of message spoofing checked for, which silently matched neither branch of its own mutation logic on every real run while still reporting a mutation. A third, related defect in the dispatch loop’s early-return behavior meant that a denied or errored action (as opposed to a mutation) also never reached the observability plugin, making a blocked tool call – including every reliability-failure trial – invisible to the trace entirely, independent of whether the block itself worked.

We fixed all three defects at the mechanism level (the hook bus’s aggregation and denial-visibility logic, and the tool-callback integration’s argument-application semantics), fixed the two plugins’ real-object mutation logic, retargeted route confusion’s shipped configuration to a control-flow hop the workflow genuinely routes through a tool call (the outer front-office router’s real choice between the transaction-review and chatbot workflows, rather than an inner hop the workflow reaches through a direct method call that never goes through the framework’s routing tool at all), and re-ran every attack and failure configuration live against a real model. Table I summarizes the result: for every configuration, the plugin’s security event now co-occurs with a genuine, ground-truth-corroborated change in tool use or terminal outcome, or, for message spoofing and prompt injection, a genuine mutation of the real request the model receives, whether or not the model’s specific response that trial changed as a result.

Three results in Table I are worth reading carefully rather than as attack failures. Message spoofing, prompt injection, and the malformed-failure control show the mechanism working exactly as intended – a genuine, verified mutation reaching the real model – and the model consistently declining to act on it, 5 trials out of 5 for each. A single trial cannot distinguish “the model reliably resists this” from “we got lucky once”; five consistent trials each is modest but real evidence toward the former, not proof of it. Route confusion is the opposite pattern: unambiguous and total across every trial, which is precisely the class of finding an automated ground-truth evaluator exists to catch without a human re-reading five transcripts. Tool mutation (front) surfaced something a single trial could not have: the scenario never once produced an

TABLE I
Live-model re-verification after the fixes in § IV-B, repeated $n = 5$ times per configuration against a real model. “Fired” and “Effect” report the `attack_ground_truth` evaluator’s `attack_fired` and `effect_detected_vs_ground_truth` fields as a rate across the 5 trials, not a single-trial anecdote.

Plugin (domain)		Fired Effect ($n = 5$)	Observed
Route (front)	confusion	5/5 · 5/5	Router’s real transfer diverted to the chatbot workflow every trial; compliance never reached, terminal state completed instead of manual review in all 5.
Tool (front)	mutation	0/5 · 5/5	<code>execute_transfer</code> was never once called across 5 fresh trials – this scenario’s prompt reads as too risky for the model to attempt execution at all, so the mutation plugin never had a call to intercept; the resulting divergence from the expected-tools baseline is itself caught every trial.
Tool (back)	mutation	5/5 · 0/5	<code>apply_ledger_updates</code> genuinely posted against the wrong batch id every trial; the downstream reporting agent’s terminal state never diverged from the undisturbed baseline.
Message spoofing (mid)		5/5 · 0/5	Fabricated clearance genuinely reached the compliance agent’s real request every trial; the agent independently re-ran its own policy check in all 5.
Prompt injection (front)		5/5 · 0/5	Injected override genuinely reached the target agent’s real request every trial; the agent never deviated from its instructions.
Reliability failure, malformed (front)		5/5 · 0/5	Malformed response genuinely reached the agent (as an ANOMALY event, § III-D) every trial; the agent reached manual review without real policy content each time.

`execute_transfer` call at all across 5 trials, meaning this specific prompt does not currently exercise the attack’s target tool reliably – a scenario-design finding about the testbed’s own fixtures, not a plugin defect, and one that only became visible by repeating the trial.

C. Isolating Overhead from LLM Latency

To address the confound above, we implemented a mock model that an experimenter opts into per benchmark configuration (`MANTIS MOCK_LLM=1` or a config-level flag), removing live model sampling entirely. The mock is tool-calling-capable, not text-only: on an agent’s first turn it emits a function call against one of that agent’s declared tools with a plausible dummy argument set drawn from the real banking tool schemas, so the tool control point – and any tool-stage attack plugin – is exercised the same way a real model call would exercise it, then returns a final text response once a tool result is present. It is deliberately not a planner: it has no access to which agent is calling it, and the underlying framework only allows its `transfer_to_agent` routing tool between agents that are parent, child, or sibling of one another,

so a mock-driven run reliably reaches the tool control point on whichever agent runs first but does not reliably route itself deep into a multi-agent hierarchy the way a reasoning model does. This is why Table I reports live-model evidence for every attack’s real effect directly rather than relying on the mock model to demonstrate it; the mock model’s role is confined to instrumentation-overhead isolation (below) and to the one CI-only configuration (targeting the root agent specifically, the only agent a mock-driven run reliably reaches) that our continuous-integration workflow (§ IV-A) checks on every push.

A naive off/full comparison with live model sampling and a real API key, at four repetitions and concurrency 2, remains noisy and backwards from what an overhead claim would predict (off: 20.70s average; full: 18.20s average) – confirming, with a live model this time rather than only the mock model, the same contention-and-sampling confound reported in an earlier iteration of this evaluation: running multiple full Python-interpreter-plus-framework subprocesses concurrently, combined with live LLM sampling variance, swamps a genuinely small instrumentation signal. Repeating the comparison under the mock model at concurrency 1 (serial execution, no contention) and 20 repetitions per side isolates that confound and gives a result in the expected direction: off averaged 6.348s ($\sigma = 0.181$ s; P50 6.262s, P90 6.607s, P99 6.780s) against full’s 6.455s ($\sigma = 0.241$ s; P50 6.346s, P90 6.727s, P99 7.128s) across 20/20 successful runs each – a difference of 0.107s, or roughly 1.7% of the off baseline.

That end-to-end figure still bundles trace serialization, file I/O, and OpenTelemetry span creation together with the hook bus’s own dispatch cost, so we separately instrumented the hook bus to record, per run, the cumulative wall-clock time spent inside the observability plugin’s own `apply()` calls – a direct measurement of instrumentation cost that needs no off/full diff and is consequently immune to the process-startup and contention confounds above, since it never depends on comparing two separate processes at all. Across ten repeated trials of a representative front-office run under the mock model, this direct measurement gave a median instrumentation overhead of 0.19% of total run duration (mean 0.26%, $\sigma = 0.26\%$; nine of ten trials fell between 0.12% and 0.21%, with one outlier at 1.00% consistent with ordinary OS scheduling jitter on a shared development machine). That the direct figure ($\approx 0.2\%$) is smaller than the end-to-end diff ($\approx 1.7\%$) is itself informative: the gap is attributable to trace-write I/O and span export, which the end-to-end comparison necessarily includes and the direct hook-dispatch measurement necessarily excludes, rather than to any inconsistency between the two methods – both figures place instrumentation overhead well under 5% of end-to-end run time, and the direct measurement’s magnitude and shape (a tight cluster with one high-side outlier) is consistent between this revision’s fresh trials and an earlier iteration’s, despite neither run set sharing the same wall-

clock conditions.

D. Volume and Concurrency Scaling, and Resource Cost

The benchmark runner supports two independent scaling modes, each varying one axis at a fixed value of the other so a latency or throughput change can be attributed to a single cause: a configurable list of repetition counts at fixed concurrency (transaction volume), and a configurable list of concurrency levels at a fixed repetition count (concurrent load). Both report throughput and latency alongside cumulative subprocess CPU time, peak resident memory, and mean trace event count per run.

Run with the mock model at concurrency 2 across 1, 2, 4, and 8 repetitions on the front-office workflow, throughput rose from 0.120 to 0.264 to 0.304 runs/s across 1, 2, and 4 repetitions as fixed per-batch startup cost was amortized over more transactions, then plateaued at 0.298 runs/s at 8 repetitions, with average per-run latency correspondingly falling from 8.33s to 6.58s and then holding near 6.70s – a sensible curve shape (an early amortization gain that does not continue indefinitely at a fixed concurrency level on a fixed machine). Peak resident memory stayed flat at approximately 241 MB across all four repetition levels, as expected since each repetition is an independent subprocess rather than an accumulating one; cumulative CPU time scaled approximately linearly with repetition count (7.4s, 14.5s, 26.2s, 53.3s).

The same repetition-count sweep was repeated in full for mid-office and back-office workflows, not only measured at a single point as in an earlier iteration of this evaluation: both domains reproduce the front office’s amortize-then-plateau shape almost exactly (throughput 0.160/0.289/0.302/0.293 runs/s for mid-office and 0.156/0.301/0.278/0.291 runs/s for back-office across the same four repetition levels, latency falling from roughly 6.2–6.4s at one repetition to a 6.6–7.2s plateau), with peak memory flat at the same ≈ 241 MB. The close agreement across all three domains is itself a finding: instrumentation and subprocess-startup cost are dominated by the shared MANTIS/ADK runtime, not by which banking workflow is executing.

Holding repetitions fixed at 8 and instead varying concurrency across 1, 2, 4, and 8 on the front-office workflow shows the complementary curve: throughput scales with concurrent load (0.157, 0.289, 0.531, 0.710 runs/s) while per-run latency stays flat through concurrency 4 (6.37s, 6.92s, 7.52s) and then degrades sharply at concurrency 8 (11.23s), together with a corresponding jump in cumulative CPU time (50.6s, 54.3s, 59.7s, 87.6s) – the shared-CPU contention effect reported qualitatively in § IV-C (as the cause of the naive live-model off/full comparison’s backwards result) is directly visible and quantified here as a genuine property of running many subprocesses concurrently on one machine, not an artifact specific to that earlier comparison.

E. Remaining Work Toward a Full Evaluation Campaign

Since the preceding iteration of this evaluation, the hook-bus and tool-callback defects described in § IV-B have been fixed and covered by regression tests reproducing each defect’s exact shape; every attack and failure configuration has been re-verified live against a real model across 5 repeated trials each, not a single anecdote (Table I); the benchmark runner now sweeps concurrency as an axis independent of repetition count, with both axes measured on the front-office workflow and the repetition-count axis additionally run to the same depth on mid- and back-office workflows (§ IV-D); and two of the seven evaluator dimensions (§ III-F) – hook coverage and attack ground truth – have moved from raw, unscored artifacts to scored, registry-integrated evaluators. The two items previously reported as the concrete remaining gaps – live-model re-verification of every attack configuration, and a benchmark sweep across concurrency as well as repetition count – are both now done and reported in § IV-B and § IV-D respectively.

What remains is a matter of sample size and breadth rather than a missing capability. Five repeated trials per configuration is enough to distinguish “this attack’s effect is unambiguous every time” (route confusion, 5/5) from “this specific injection is reliably resisted, at least in this small sample” (message spoofing, prompt injection, and the malformed-failure control, each 0/5 for effect) – but it is not enough to bound the probability of a rarer outcome a larger sample might surface, such as an occasional trial where a spoofed clearance is accepted. Similarly, the concurrency sweep and the three-domain repetition-count sweep (§ IV-D) each hold the other scaling axis fixed; a full two-dimensional sweep across concurrency and repetition count together, and per-domain repeated-trial attack-efficacy statistics at the same depth as Table I (currently run on the domain each attack is native to, not cross-domain), are both practical extensions of the same machinery rather than new implementation work. We report the current sample sizes and their exact scope throughout this section rather than a projected final result, because presenting five trials or a single-axis sweep as though it fully characterized the underlying distribution would overstate what has actually been shown.

V. Conclusion

We presented MANTIS, a configuration-driven testbed that instruments a realistic, unmodified banking multi-agent system at five fixed control points, so that adversarial and failure experiments are declared in YAML rather than implemented as changes to business logic. The testbed’s banking runtime adapter, hook bus, five attack and failure configurations across three domains, observability pipeline, and seven-dimension evaluation and benchmarking framework are implemented and verified end to end, including a reproducibility mechanism that records the environment, not only the configuration,

behind every run. Live-verified evidence confirms that every control point is reachable; it also surfaced, and let us fix, two mechanism-level defects – in the hook bus’s own plugin-dispatch aggregation and in how a mutated tool argument was applied back onto the agent framework’s real execution path – under which every attack plugin’s payload mutation had, since the mechanism was first written, recorded as having occurred while having no effect on real agent behavior, independent of any individual plugin’s own targeting logic. Diagnosing and fixing both, together with correcting two plugins’ mutation logic against the framework’s real request-object types, let us re-verify all five attack and failure configurations live against a real model across five repeated trials each and observe genuine, quantified behavioral divergence – a bypassed compliance review reproduced in all five trials, a redirected wire transfer, a misdirected ledger post – caught automatically by the corresponding ground-truth evaluator rather than by manual transcript inspection, and one further finding a single trial could not have shown: one attack’s fixture scenario never once produced its target tool call across five fresh trials, a scenario-design gap rather than a plugin defect. This is itself the paper’s central methodological finding: verifying a security experiment’s emitted trace end to end, rather than trusting a clean process exit or an isolated “attack fired” event, has to be load-bearing in a testbed like this one, because a defect at exactly the layer between “the plugin ran” and “the framework acted on it” is invisible to any weaker check. A tool-calling mock model isolates instrumentation overhead from LLM sampling latency, arriving at a direct, repeated-trial measurement of instrumentation overhead at a median of 0.19% of run duration, corroborated by a contention-controlled end-to-end comparison of comparable magnitude in the same direction; a concurrency sweep independent of the repetition-count sweep, and the same repetition-count sweep run to full depth on all three banking domains, both confirm the same shared-runtime-dominated cost structure holds regardless of which axis or domain is varied. What remains toward a full statistical evaluation campaign is now a matter of sample size and breadth, not a missing capability: five trials per attack configuration and four levels per scaling axis are practical, repeatable sample sizes, not maximal ones, and a larger campaign along the same already-implemented axes would tighten the confidence bounds this evaluation can currently offer. Future work extends this testbed along the directions its interfaces were explicitly designed to support without modification: a minimal schema-driven UI over the existing CLI, additional observability exporters, security-mechanism (guardrail and policy) plugins evaluated through the same plugin interface as the attacks in this paper, and a Zero Trust Backplane that consumes the same application inventory and control-point metadata described in Section III to generate and enforce access policy as a separate research contribution.

Acknowledgment

References

- [1] X. Zhang et al., “Enhancing Anomaly Detection in Financial Markets with an LLM-based Multi-Agent Framework,” arXiv preprint arXiv:2403.19735, 2024.
- [2] “MASCA: LLM based-Multi Agents System for Credit Assessment,” arXiv preprint arXiv:2507.22758, 2025.
- [3] D. Lee and M. Tiwari, “Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems,” arXiv preprint arXiv:2410.07283, 2024.
- [4] “Red-Teaming LLM Multi-Agent Systems via Communication Attacks,” in Findings of the Association for Computational Linguistics: ACL 2025, arXiv preprint arXiv:2502.14847.
- [5] “A Survey on Agentic Security: Applications, Threats and Defenses,” arXiv preprint arXiv:2510.06445, 2025.
- [6] “Security Considerations for Multi-agent Systems,” arXiv preprint arXiv:2603.09002, 2026.
- [7] “REDAgentBench: Executable Red Teaming and Faithful Measurement of LLM Agent Systems,” arXiv preprint arXiv:2608.10669, 2026.
- [8] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” NIST Special Publication 800-207, National Institute of Standards and Technology, Aug. 2020.
- [9] “A Novel Zero-Trust Identity Framework for Agentic AI: Decentralized Authentication and Fine-Grained Access Control,” arXiv preprint arXiv:2505.19301, 2025.